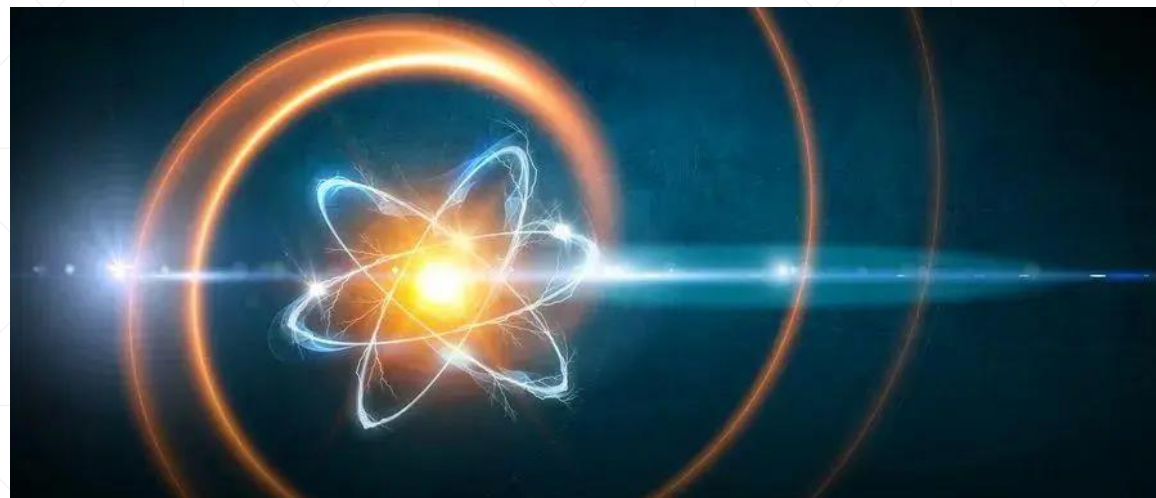


CompletableFuture之二



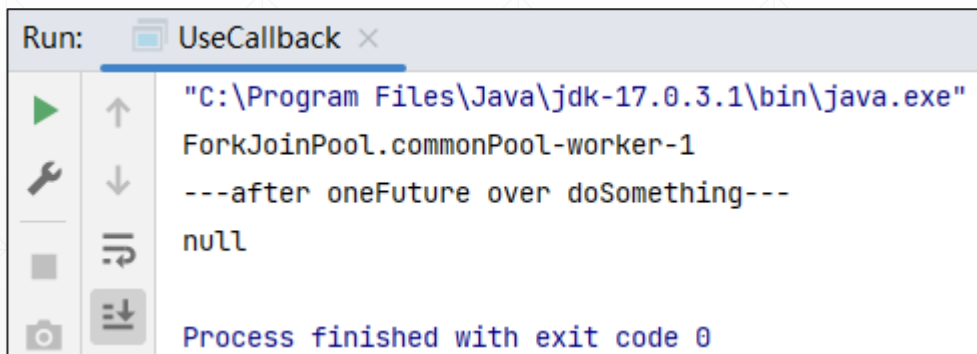
“回调”的妙用

北京理工大学计算机学院
金旭亮

最典型的异步编程模式，是使用“回调”.....

使用回调-1

基于**thenRun**实现异步任务A，执行完毕后，激活异步任务B执行，需要注意的是，这种方式激活的异步任务B是拿不到任务A的执行结果的，除非任务A结果放在一个全局变量中。



```
Run: UseCallback x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
ForkJoinPool.commonPool-worker-1
---after oneFuture over doSomething---
null
Process finished with exit code 0
```

UseCallback.java

```
private static void useThenRun() {
    // 创建异步任务, 并返回future
    var taskA : CompletableFuture<Void> = CompletableFuture.runAsync(
        () -> {
            var threadName : String = Thread.currentThread().getName();
            // 返回计算结果
            System.out.println(threadName+"计算完成, 结果为: Hello.");
        });

    // 在future上施加事件, 当future计算完成后回调该事件, 并返回新future
    var taskB : CompletableFuture<Void> = taskA.thenRun(
        () -> {
            var threadName : String = Thread.currentThread().getName();
            //当oneFuture任务计算完成后做一件事情
            System.out.println(threadName+"执行计算完成后被回调的方法---");
        });

    //同步等待对应的任务完成
    taskB.join();
}
```

使用回调-2

使用**thenApply()**方法可以取回上一步骤的结果，它在上一步骤的执行线程中执行。

```
private static void useThenApply()
    throws InterruptedException, ExecutionException {
    // 创建异步任务
    CompletableFuture<String> stepOne = CompletableFuture.supplyAsync(
        () -> "这是第一步的结果");
    // 第一步完成后，继续进行第二步
    CompletableFuture<String> stepTwo = stepOne.thenApply(
        prevResult -> {
            //返回加工后结果
            return prevResult + "\n这是第二步的结果";
        });
    // 3.同步等待twoFuture对应的任务完成，并获取结果
    System.out.println(stepTwo.get());
}
```

使用回调-4

```
private static void useThenAccept() {  
    var future : CompletableFuture<Integer> = CompletableFuture  
        .supplyAsync(() -> {  
            var threadName : String = Thread.currentThread().getName();  
            System.out.println(threadName + "生成随机数。");  
            return new Random().nextInt();  
        });  
  
    //使用thenAcceptAsync, 可接收前一个任务的结果  
    future.thenAcceptAsync(result -> {  
        var threadName : String = Thread.currentThread().getName();  
        System.out.println(threadName + "执行异步回调函数, 得到的随机数为: " + result);  
    }).thenRun(() -> {  
        var threadName : String = Thread.currentThread().getName();  
        System.out.println(threadName + "执行同步回调函数执行");  
    }).join();  
}
```

如果需要**异步**接收前一个任务生成的结果, 可以使用**thenAcceptAsync()**, 它的参数就是前一步骤传来的结果, 注意这是一个异步方法, 所以要join等待它的完成。

Run: CompletableFuturesDemo X

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files  
ForkJoinPool.commonPool-worker-1生成随机数。  
ForkJoinPool.commonPool-worker-1执行同步回调函数执行  
ForkJoinPool.commonPool-worker-1执行异步回调函数, 得到的随机数为: 720556091  
  
Process finished with exit code 0
```

使用回调-4

另一种方式是使用`whenComplete()`运行回调函数，这么做的好处在于一可以获取前一个任务的执行结果，二是可以写代码捕获异常。

```
private static void useWhenComplete() {  
    // 创建异步任务，并返回future  
    CompletableFuture<String> future = CompletableFuture.supplyAsync(  
        () -> "这是第一个任务的结果。");  
    future.whenComplete((result, throwable) -> {  
        -----  
        // 如果没有异常，打印异步任务结果  
        if (null == throwable) {  
            System.out.println(result);  
        } else {  
            //打印异常信息  
            System.out.println(throwable.getLocalizedMessage());  
        }  
    }).join();  
}
```

回调应用实例：实现数据转换

//完成数据转换工作

```
private static int doubleValue(int value) {  
    return value * 2;  
}
```

```
private static void transformResult() {  
    var future : CompletableFuture<Integer> = CompletableFuture.supplyAsync(() -> 100);  
    //使用thenApply转换数据, 使用thenAccept接收最后的处理结果  
    future.thenApply(CompletableFutureDemo::doubleValue)  
        .thenAccept(System.out::println);  
}
```

应用实例：函数复合

```
private static CompletableFuture<String> asyncTask1() {  
    return CompletableFuture.supplyAsync(() -> "第一步的结果");  
}  
  
private static CompletableFuture<String> asyncTask2(String prevResult) {  
    return CompletableFuture.supplyAsync(() -> {  
        return "收到: " + prevResult + ",继续处理, 得到第二步的结果";  
    });  
}  
  
private static void useComposeFunc(){  
    //先执行第一个异步任务, 接着是第二个, 它接收第一个异步任务的结果, 最后, 第二个结果被输出  
    asyncTask1()  
        .thenCompose(CompletableFuturesDemo::asyncTask2)  
        .thenAccept(System.out::println).join();  
}
```

ǎ ĝ y

UseComposeAndCombine.java

回调应用实例：函数组合

```
private static void useCombineAsyncFunc() {  
    var first : CompletableFuture<Integer> = CompletableFuture.supplyAsync(() -> "100")  
        .thenApply(Integer::parseInt);  
    var second : CompletableFuture<Double> = CompletableFuture.supplyAsync(Math::random);  
    //异步计算两个函数，然后以这两个函数的返回值为参数，进行计算，得到最终结果，再输出  
    first.thenCombine(second, (firstResult, secondResult) -> {  
        return firstResult * secondResult;  
    }).thenAccept(System.out::println);  
}
```